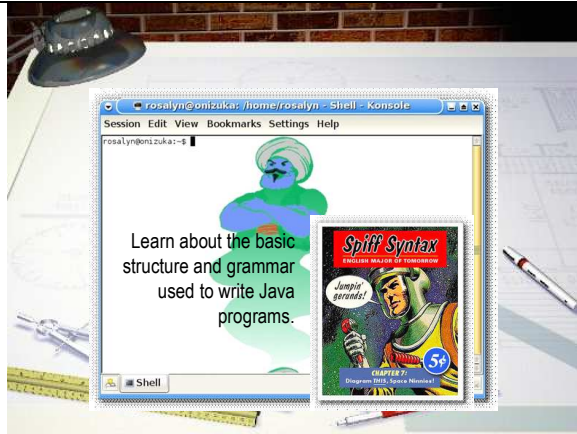




Slide 1

Java Programming 1

Lecture 2E

Java Basics

Duration: 00:01:09

Advance mode: Auto

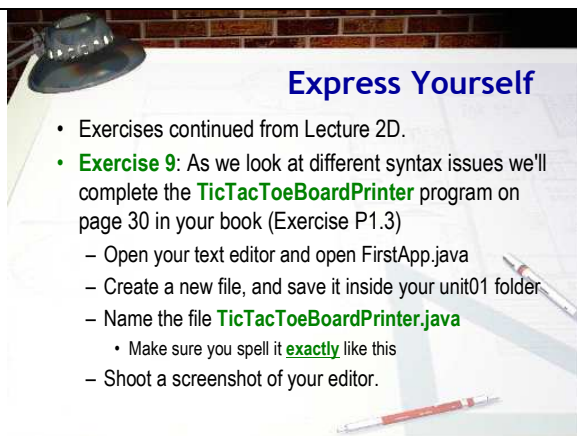
Notes:

This is the CS 170 Java Programming 1, Lecture 2E, Java Basics.

In the previous lecture, you entered, compiled, and ran the Java application, FirstApp.java, treating your computer keyboard much like Aladdin's magic lamp. You typed in the incantation, and, if you did it just right, the compiler accepted it, and your program ran.

Of course, programming really isn't magic; instead of memorizing a dusty volume of "Open Sesame", in this class, you'll learn the meaning behind each of the phrases that make up your Java programs. (I have to admit, though, that there will still be times when I'll wave my hands, and say something that sounds suspiciously like "Abracadabara". But those times will get much rarer as the semester goes on.)

This lecture covers the basic grammar or syntax that all Java programs follow. Before you begin this lesson you should have installed the Java Development Kit, and know how to editing compile and run a Java application and a Java applet.



Slide 2

Express Yourself

Duration: 00:01:37

Advance mode: Auto

Notes:

In this lecture we'll continue the exercises from Lecture 2D, "Java Mechanics". Put your answers in the same in-class exercise document.

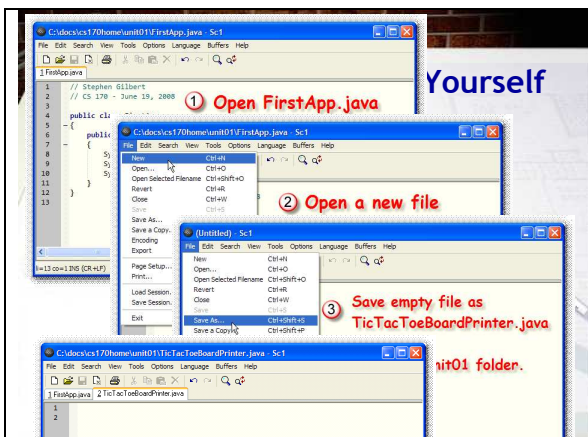
For Exercise 9, we're going to start a new program, similar to one of the assignments that you'll complete for your homework. The program we'll write is the TicTacToeBoardPrinter program on page 30 of your textbook. If you are using WileyPlus, you'll find it in the exercises at the end of Chapter 1. The exercise is number P1.3.

On this slide I'll go over the steps. The next slide has screenshots that might help you if you get stuck.

Open your text editor and open FirstApp.java (from Lecture 2D). Now, create a new file (using File New from the menu), and save it inside your unit01 folder. Note that SciTE allows you to have multiple files open at the same time; you switch files by clicking the file's tab.

Name the file TicTacToeBoardPrinter.java. Make sure you spell it exactly like this, including all capitalization. If you capitalize it incorrectly, it won't work when you try to submit it for grading. (Of course, since this is an in-class exercise, it won't be graded, but you'll want to know how to do it correctly, so you don't have any problems submitting your homework, which will be graded.)

Once you've done all of that, shoot a screen-shot of your editor, so that I can see the tabs for both files.



Slide 3
Express Yourself
 Duration: 00:00:32
 Advance mode: Auto

Notes:

Here are some screen-shots of the process on the previous page.

1. Open FirstApp.java from Lecture 2D
2. Open a new file
3. Save the empty file in your unit01 folder as TicTacToeBoardPrinter.java, making sure you spell it correctly.

Here's what the top of your screen-shot should look like. Notice the tabs for each file.

- Every Java program is a **class definition**
 - Class definition is a blueprint to construct objects
 - The basic class structure looks like this:

```

... some stuff
public class SomeClassName
{
    ... some more stuff
}
  
```

- Must be stored in **SomeClassName.java**
- Only one **public** class per file

Slide 4
Program Organization
 Duration: 00:01:48
 Advance mode: Auto

Notes:

In Java, every program is a class definition. So, what does that mean.

A class definition is the blueprint that is used to construct objects, the basic building-blocks of object-oriented programs. We'll talk a lot more about objects and classes in Chapter 2 and 3. In the case of an Java applet or application, you can think of that object as a "program object." Now, not every object-oriented language works like this. In C++, for instance, every program consists of a main() function that may, or may not create different kinds of objects. In Java, there are no separate program entities; there are only classes and objects. (Later, though, we'll look at the different Program classes that are part of the ACM libraries.)

So, what does a class definition look like. The basic structure for every Java program looks like this:

```

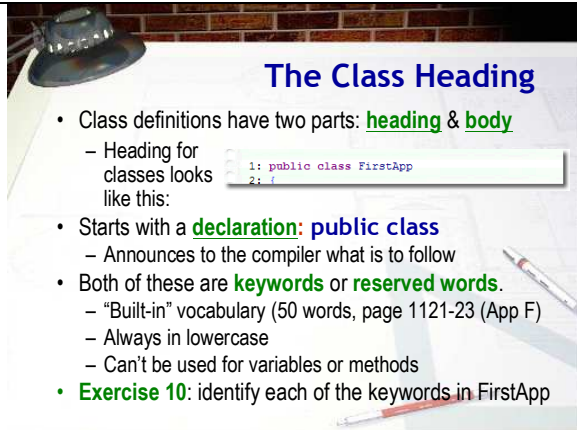
...some stuff
public class SomeClassName
{
    ... some more stuff
}
  
```

You have some stuff (that's a technical programming term, of course), then the line public class SomeClassName, followed by a set of curly braces. Inside the curly braces you have some more stuff. Let's look at some of the details.

You can choose any name you want for the word SomeClassName, but whatever name you choose, the file name for your program must be match the class name selected exactly. In this case, the definition would have to be stored inside the file named SomeClassName.java. Each Java source code file can contain only one public class definition. (Later you'll see that we can define other helper classes in the same file, but they cannot be public classes.)

Note that this is different from many other programming languages where the program name and the name of the file

are not related. In Java, the name of the file and the name of the program must be identical (including case). The file name and the public class it contains must match exactly.



The Class Heading

- Class definitions have two parts: **heading & body**
 - Heading for classes looks like this:

```
1: public class FirstApp
2: {
```
- Starts with a **declaration: public class**
 - Announces to the compiler what is to follow
- Both of these are **keywords** or **reserved words**.
 - "Built-in" vocabulary (50 words, page 1121-23 (App F))
 - Always in lowercase
 - Can't be used for variables or methods
- **Exercise 10:** identify each of the keywords in FirstApp

Slide 5

The Class Heading

Duration: 00:01:39

Advance mode: Auto

Notes:

A class definition has two parts: a class header and a class body. The header "declares" or "announces" the class for the compiler; think of it as a formal introduction. We'll come back to the body shortly, but first, let's take a closer look at the class header.

The class header for FirstApp, on line 1, looks like this:

The first part of the header is the declaration public class. The class declaration simply says to the compiler:

"May I introduce a class definition?"

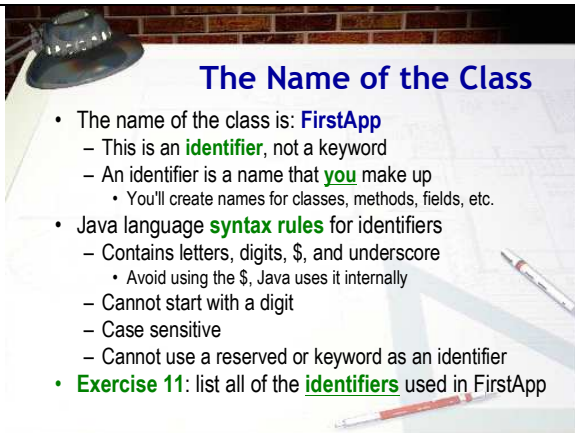
so the compiler knows how to interpret the words that follow.

Both public and class are two of the fifty-or-so Java keywords. Keywords, (also sometimes called reserved words), are the built-in vocabulary of a language. In Java:

- all keywords are lowercase
- you cannot use keywords as identifiers, (names you make up and then use to describe classes, objects, methods, and attributes)
- You must spell the keywords correctly and use them in the correct context. If you use a keyword inappropriately, or if you misspell it, you will generate a syntax error: a violation of Java's internal rules of grammar.

You'll find a list of the Java keywords in Appendix F of your textbook (Pages 1121-23 in the printed version.)

Before we go any further, for Exercise 10, take a look at FirstApp.java and identify each of the keywords you find there. Type the list into your exercise document.

A presentation slide titled "The Name of the Class" with a background image of a desk with a lamp and papers. The slide contains a bulleted list of rules for class names in Java.

- The name of the class is: **FirstApp**
 - This is an **identifier**, not a keyword
 - An identifier is a name that **you** make up
 - You'll create names for classes, methods, fields, etc.
- Java language **syntax rules** for identifiers
 - Contains letters, digits, \$, and underscore
 - Avoid using the \$, Java uses it internally
 - Cannot start with a digit
 - Case sensitive
 - Cannot use a reserved or keyword as an identifier
- **Exercise 11**: list all of the **identifiers** used in FirstApp

Slide 6

The Name of the Class

Duration: 00:02:31

Advance mode: Auto

Notes:

When you write the declaration `public class`, this puts the Java compiler into such a state of anticipation, that it won't be satisfied unless you tell it exactly what class is about to be described. Thus, the next part of the class header must be the class name. Here we've named our class `FirstApp`. (Remember, also, that this class must be located in a file that matches the class name exactly, and that has the extension `.java`.)

While `public` and `class` are keywords, the word `FirstApp` is a different kind of beast; this is called an identifier. An identifier is a name that you make up to label classes, attributes, methods, and variables. You can use any name you like, as long as you follow a few simple rules:

1. you can use letters, digits, and the underscore character. (You can also use the dollar sign, but you should avoid doing so, since Java uses it internally for some auto-generated class names.)
2. your identifier cannot start with a digit
3. identifiers are case sensitive
4. you cannot use keywords as identifiers

Remember, unlike keywords, (which are all lowercase), you may use uppercase as well as lowercase in your identifiers. But, the identifiers `cat`, `Cat`, and `CAT` are not the same! If you've programmed in another language, such as C, Pascal, or Visual Basic, these rules don't seem too different. The biggest difference (when it comes to identifiers) between Java and these other languages is not readily apparent, but pops to the surface when you ask the question, "What's a letter?"

Unlike these other languages Java isn't limited to using the ASCII character set (the most widely used computer-character encoding scheme). Instead, it uses Unicode, the international character set that provides some 65,000 different characters. So, although Java identifiers are limited to characters and digits, those characters and digits can be any valid Unicode character or digit. (In other words, you can use Japanese, Russian, or Sanscrit characters when creating your identifiers.)

For Exercise 11, turn back to `FirstApp.java`, and list all of the identifiers you find there in your exercise document.

Method Basics

- Class body is delimited by braces `{ }` (begin...end)
- Inside the class body, you'll define two things:
 - Attributes** -- the data fields used to store object state
 - Methods** -- where the actions take place in an object
- Methods are used in two places in your programs:
 - When you **define** or specify instructions to perform
 - When you **use, invoke** or **call** a method
- Exercise 12:** add the necessary code to create the `TicTacToeBoardPrinter` class and compile

```

1 public class FirstApp
2 {
3     // Attributes
4     // Methods
5 }

```

Slide 7

Method Basics

Duration: 00:02:55

Advance mode: Auto

Notes:

The class body, (the portion of the class in braces `{ }`), is where the attributes and methods for the class are defined. In Java, braces `{ }` are used to mark the beginning and end of the class body. Braces are also used to surround (or delimit) the body of a method, as well as the bodies of loops and selection statements. Just think of braces as punctuation where the opening or right-facing brace says: "this is the beginning of a section", and the closing or left-facing brace says: "this is the end of a section". In programming, a delimited "section" of code like this is called a block. Braces, along with indentation, allow you to quickly see and understand the general structure or "shape" of a program. To facilitate this, every time you use a set of braces, the "stuff" inside the braces should be indented, by one tab stop, as well.

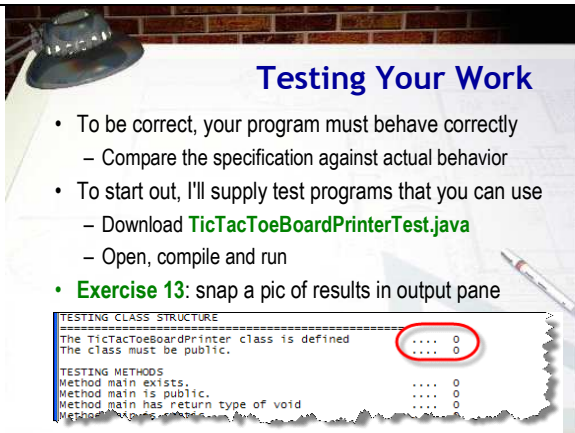
Inside the braces that are used to delimit the class body, we're going to specify two things.

- the data elements that our program will work with. These are called the object attributes and they'll be used to store the object's state. We'll talk more about object state in Chapter 2, and you'll learn to define these data fields in Chapter 3.
- the class methods that specify how each object is supposed to act. Methods are where the actions take place in your class. We'll learn how to write a method in this lesson.

We actually use methods in two places in our programs. Let me give you an analogy. Many of us keep a TO-DO list of the tasks we need to perform each day, perhaps using an online calendar or a day-planner, or just on the back of an envelope. These steps are like a method definition, which describes a set of actions to be performed at some later time.

When a new day comes around, you open your day-planner, and start executing the tasks—you go shopping, go to school, go to the doctors, and so on. Similarly, when we tell our computer to carry out the actions in the method definition, we are using, invoking or calling the method.

We'll see how to do both of these things shortly. Before we get to defining and calling methods, let's see if you can remember what we've covered on the last few slides. Let's leave `FirstApp` behind and switch to the `TicTacToeBoardPrinter` file, and add the necessary code to create the `TicTacToeBoardPrinter` class. Save your file once you've finished, and make sure that your program compiles. If you compile inside SciTE, shoot a screen-shot showing both your code and the output window with a successful compile. If you compile from the command-line, show me two screen-shots: one of your source code and one of the command-line window with a successful compile.



Testing Your Work

- To be correct, your program must behave correctly
 - Compare the specification against actual behavior
- To start out, I'll supply test programs that you can use
 - Download **TicTacToeBoardPrinterTest.java**
 - Open, compile and run
- Exercise 13:** snap a pic of results in output pane

```

TESTING CLASS STRUCTURE
=====
The TicTacToeBoardPrinter class is defined      .... 0
The class must be public.                      .... 0

TESTING METHODS
=====
Method main exists.                            .... 0
Method main is public.                        .... 0
Method main has return type of void           .... 0
Method exists and has correct return type      .... 0
  
```

Slide 8

Testing Your Work

Duration: 00:02:08

Advance mode: Auto

Notes:

Even though your code compiles, it's possible that you haven't done it correctly: that is, it's possible that the class you defined doesn't meet the specification you were asked to follow. The javac compiler can only tell if your program is grammatically correct; it doesn't have any way to tell if it is actually doing what it was supposed to do. For that, you'll need to test your work.

Testing your work is critical to writing good software, and it's important that you learn how to test your own code. To start out, though, I'll supply you with some test programs, and you'll run those. Later you'll write some test programs that you run in addition to mine. By the time we're finished with the course, you'll be writing your own testing programs and won't need mine at all.

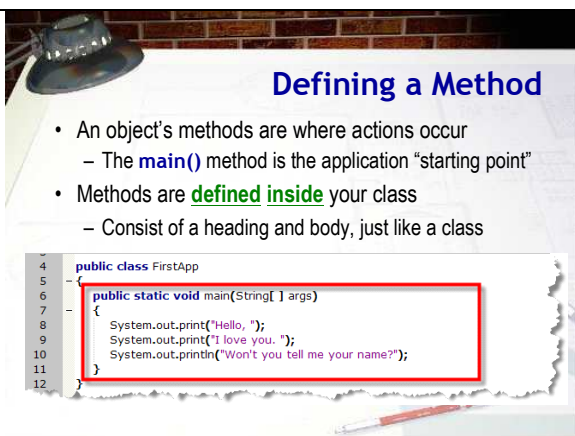
To run the test program for TicTacToeBoardPrinter, download the file TicTacToeBoardPrinterTest.java from the class Web site, and save it in the same folder where you've stored TicTacToeBoardPrinter.java. Then open the test program in your editor, compile it, and run it. Then, for Exercise 13, snap me a picture of the results in the output pane.

If you've defined the class correctly, you should see some output like this, with two points assigned for defining the class correctly.

If you left off the word public when you defined your class, you'd see something like this, with only one point for the class definition, but no points for the public modifier.

If you named your class incorrectly, or made other mistakes in the class definition, you'll get zeros for both of the first test in the CLASS STRUCTURE section, like this.

Make changes to TicTacToeBoardPrinter.java, save your work, recompile, and then run the test program again. You only have to compile the test program once, but you have to remember to save your changes to the program you're testing and recompile it before each test run. When you get the two ones, you're ready to go on to defining methods.



Defining a Method

- An object's methods are where actions occur
 - The **main()** method is the application "starting point"
- Methods are **defined inside** your class
 - Consist of a heading and body, just like a class

```

4 public class FirstApp
5 {
6     public static void main(String[] args)
7     {
8         System.out.print("Hello, ");
9         System.out.print("I love you. ");
10        System.out.println("Won't you tell me your name?");
11    }
12 }
  
```

Notes:

When you create a new class, you'll have to define the methods that give the class its distinctive behavior. If you don't define any methods, then your program just won't do anything!

Every Java application program has to define one such method, the method named main(). This method definition is placed inside of the class definition, not after it. One mistake that beginners often make is to place the class definition after the closing brace of the class itself; the error message that the Java compiler prints when you do that isn't especially helpful. Here's the main method from our FirstApp program.

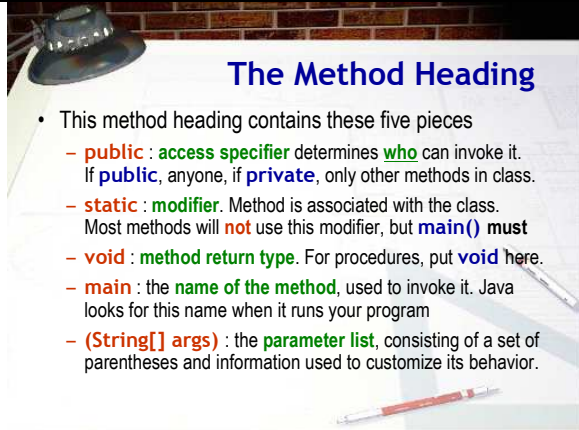
Slide 9

Defining a Method

Duration: 00:00:53

Advance mode: Auto

Notice that a method definition, like a class definition, consists of both a header, and a body, but that the method header contains different keywords than the class header.



The Method Heading

- This method heading contains these five pieces
 - **public** : **access specifier** determines **who** can invoke it. If **public**, anyone, if **private**, only other methods in class.
 - **static** : **modifier**. Method is associated with the class. Most methods will **not** use this modifier, but **main()** **must**
 - **void** : **method return type**. For procedures, put **void** here.
 - **main** : the **name of the method**, used to invoke it. Java looks for this name when it runs your program
 - **(String[] args)** : the **parameter list**, consisting of a set of parentheses and information used to customize its behavior.

Slide 10

The Method Heading

Duration: 00:02:37

Advance mode: Auto

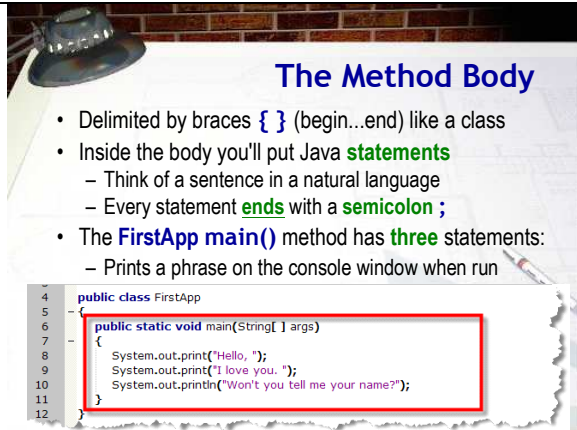
Notes:

This method header consists of these five pieces:

- **public** : This is called the access specifier and it means that objects from different classes are free to ask the FirstApp class to perform this action. If the method were private, then only other FirstApp objects would be able to invoke the method, and we couldn't run the program. You'll make almost all of your methods public.
- **static** : This is a modifier indicating that the main() method isn't associated with a particular instance of the FirstApp class, but with the class itself. This allows it to be used to start your program running, even though no FirstApp objects have been created. Most methods will not use this modifier, but the main() method must.
- **void** : The access specifier is followed by the method return type. Every method can, potentially, produce a single value. If the main() method produced a value, we'd put the type of that value (number, character, etc) here. If a method produces no value—like main()—you have to say so by using a return type of void.
- **main** : This is the name of the method. Because Java looks for a method named main() when you try to run your program, we must be careful to spell this exactly as shown. Writing Main() or MAIN() will not create an error, but your method won't be called when you try to run the program either. The Java Virtual Machine will only call the main() method, and it's very particular about the spelling.
- **(String[] args)** : Every method definition has a set of parentheses following the method name. If a method requires additional information to carry out its work, then that information is supplied as a list of parameters. The main() method takes one, single parameter, an array of String objects, that is traditionally named args. We could have named this object anything we like, because "args" is just an identifier. Using CommandLineStrings instead of args would have worked just as well; it would just require a bit more typing than when we used the name args.

In reality, it's kind of early in the course to be dropping all of

these details on you. For now, I'd simply treat the syntax of the main() method as an incantation that has to be typed in a particular manner, and leave it at that. If you understand that main is where a Java application starts running, then you're doing fine.



The Method Body

- Delimited by braces **{ }** (begin...end) like a class
- Inside the body you'll put Java **statements**
 - Think of a sentence in a natural language
 - Every statement **ends** with a **semicolon ;**
- The **FirstApp** **main()** method has **three** statements:
 - Prints a phrase on the console window when run

```
4 public class FirstApp
5 {
6     public static void main(String[] args)
7     {
8         System.out.print("Hello, ");
9         System.out.print("I love you. ");
10        System.out.println("Won't you tell me your name?");
11    }
12 }
```

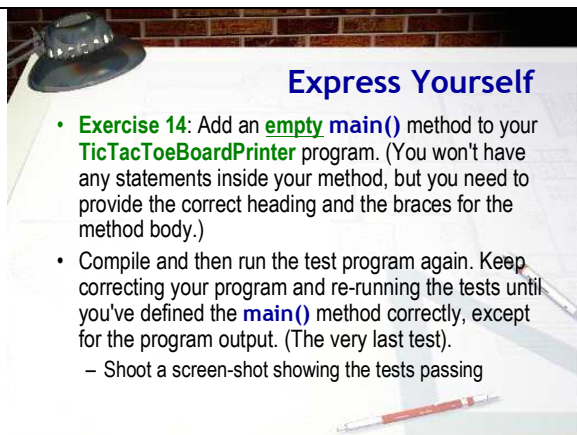
Slide 11
The Method Body
Duration: 00:00:57
Advance mode: Auto

Notes:

Like the class body, the begin and end of a method body are marked by braces.

Inside the braces are statements that perform actions or create objects. You can think of a statement like the sentences in a natural language. All of the statements in your Java program will end with a semicolon. Each time the method is activated (that is, called or invoked), all of the statements inside the method body are performed, in order, one after another. As previously mentioned, the main() method is called automatically by the JVM when the program is run.

FirstApp's main() method contains three statements that use the PrintStream object System.out object to display the phrase from the Doors album, "Hello, I love you. Won't you tell me your name?" in the console window.



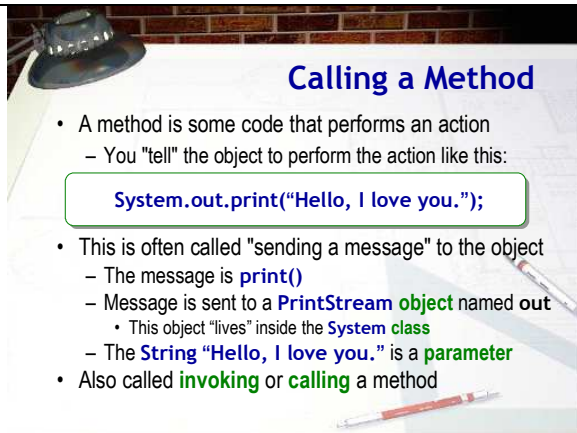
Express Yourself

- **Exercise 14:** Add an **empty main()** method to your **TicTacToeBoardPrinter** program. (You won't have any statements inside your method, but you need to provide the correct heading and the braces for the method body.)
- Compile and then run the test program again. Keep correcting your program and re-running the tests until you've defined the **main()** method correctly, except for the program output. (The very last test).
 - Shoot a screen-shot showing the tests passing

Slide 12
Express Yourself
Duration: 00:00:40
Advance mode: Auto

Notes:

OK, now let's see if you can put what you've learned into practice. For Exercise 14, add an empty main() method (that is, one with no statements in its body), to your TicTacToeBoardPrinter program. Once you've added the new statements, compile your program, and then switch over to the test program in run it. Keep correcting, saving, compiling and testing until all of the tests are correct except for the very last one, the one appearing under the TESTING CLASS BEHAVIOR section. Shoot me a screen-shot when you've finished, showing your passing tests.



Calling a Method

- A method is some code that performs an action
 - You "tell" the object to perform the action like this:


```
System.out.print("Hello, I love you.");
```
- This is often called "sending a message" to the object
 - The message is `print()`
 - Message is sent to a **PrintStream object** named `out`
 - This object "lives" inside the **System class**
 - The **String** `"Hello, I love you."` is a **parameter**
- Also called **invoking** or **calling** a method

Slide 13
Calling a Method
 Duration: 00:01:13
 Advance mode: Auto

Notes:

Now that we've looked at defining a method (the `main()`), let's look at how you call or invoke a method. A method is a named piece of code that performs an action. Usually a method is attached to a particular object, and, you can tell the object to perform an action like this:

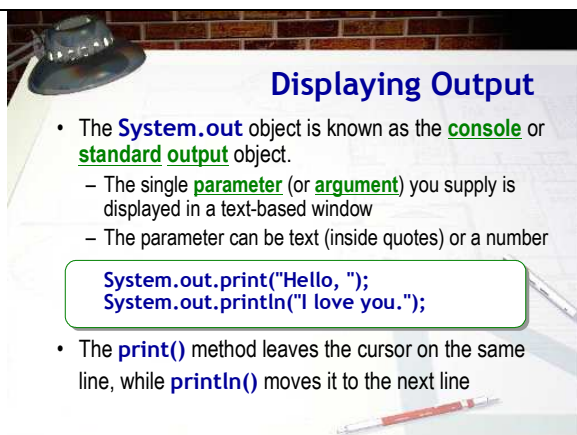
```
System.out.print("Hello, I love you.");
```

This is also often called "sending a message" to the object.

The command or message that we want to send is named `print`, and we're sending it to a `PrintStream` object named `out`, that lives in the `System` class. (We'll learn more about this object later. For right now, think of it a Genie that will accept your output and display it for you on the screen.)

The text that you want to display `"Hello, I love you."`, is passed to the method as a parameter. In Java, quoted text like this is called a `String`.

To carry out your request, the `PrintStream` object will use a method, and the method will also be named `print`. Every time you send a message to an object, that object must have a method of the same name as the message. Thus, "sending a message" and "invoking (or calling) a method", usually mean the same thing, and I'll frequently use them interchangeably.



Displaying Output

- The **System.out** object is known as the **console** or **standard output** object.
 - The single **parameter** (or **argument**) you supply is displayed in a text-based window
 - The parameter can be text (inside quotes) or a number

```
System.out.print("Hello, ");  
System.out.println("I love you.");
```

- The **print()** method leaves the cursor on the same line, while **println()** moves it to the next line

Slide 14
Displaying Output
 Duration: 00:01:17
 Advance mode: Auto

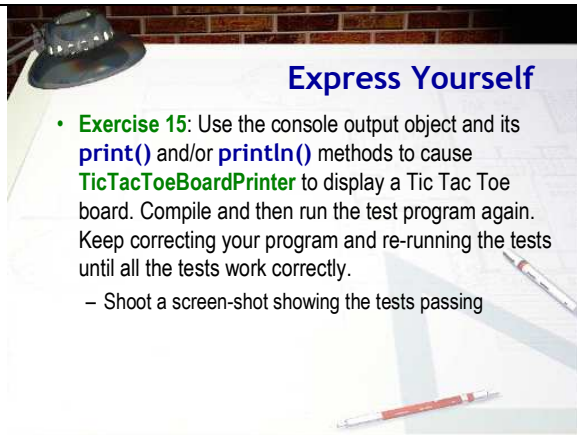
Notes:

Let's take a little closer look at this object named `System.out`. This is a predefined object, created by the JVM every time you run a program. It's also known as the console or standard output object.

For right now, we're interested in two of the actions that this object can carry out on our behalf: `print` and `println`. When we call either of these methods we need to supply it with a single piece of information (called a parameter), and the `System.out` object will take the information and display it on the screen. The parameter can be a complete sentence, as long as you remember to put double-quotes around it, or it can be a number. (Actually, it can be almost any kind of object, as we'll see later).

Here are two examples.

The difference between the `print` and `println` methods, is that `print` leaves the cursor immediately following the last character printed (that is, on the same line as the output), so that the next print or input takes place on the same line. The `println` method adds a newline character to the end of the output, so the next time `print` or `println` is called, the output appears on the next line on the screen.



Express Yourself

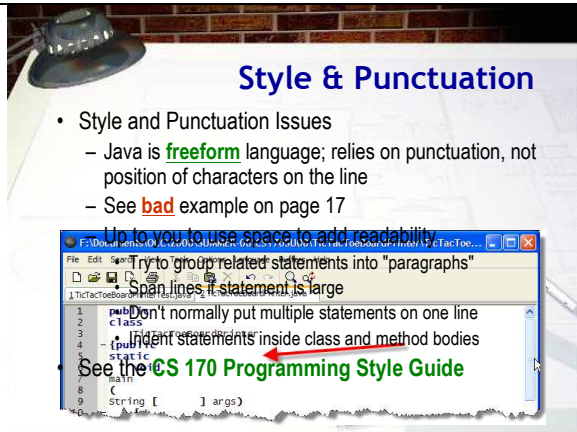
- **Exercise 15:** Use the console output object and its `print()` and/or `println()` methods to cause `TicTacToeBoardPrinter` to display a Tic Tac Toe board. Compile and then run the test program again. Keep correcting your program and re-running the tests until all the tests work correctly.
 - Shoot a screen-shot showing the tests passing

Slide 15
Express Yourself
 Duration: 00:00:39
 Advance mode: Auto

Notes:

OK, its your turn again. Use `System.out` and the `print` and/or `println` methods to display a tic tac toe board on the screen. For the corners use the plus sign, for the horizontal lines use the minus sign and for the vertical lines, use the vertical bar character which appears on the same key as the backslash, right above the Enter key.

Save and compile your program, and then run the test program again. Keep going until you get 100% of the tests passing. Shoot me a screenshot showing all tests passing.



Style & Punctuation

- Style and Punctuation Issues
 - Java is **freeform** language; relies on punctuation, not position of characters on the line
 - See **bad** example on page 17
- Up to you to use space to add readability
 - Try to group related statements into "paragraphs"
 - Span lines if statement is large
 - Don't normally put multiple statements on one line
 - Indent statements inside class and method bodies
- See the **CS 170 Programming Style Guide**

Slide 16
Style & Punctuation
 Duration: 00:01:53
 Advance mode: Auto

Notes:

Before we finish up this lesson, let's look at a couple of style issues you'll need to understand when you write your own Java programs. Java is a free-form language. That means that statements may span several lines, like this, or may be run all together like the bad example shown on page 17 of your textbook.

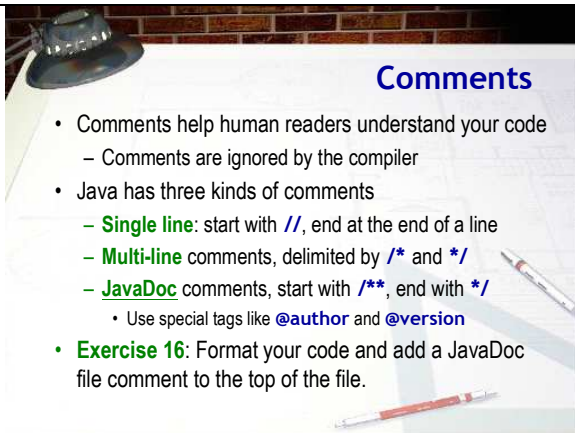
You probably don't want to write code like this, however, and you certainly don't want to read code written using this style, but it's up to you to use space wisely. You should use whitespace such as blank lines and indenting to improve the readability of your programs, not reduce the readability.

Try to group related statements together into "paragraphs", and separate such groups from unrelated groups of statement by inserting blank lines and comments.

Make sure you don't have lines longer than 80 characters. If you do, break the line into pieces and place them on multiple lines. When you do this, make sure you indent the spanned lines by one tab-stop, so it's obvious that the line is a continuation of the one previous to it.

Finally, make sure you indent all of the statements inside each block by one tab stop (4 spaces). This includes all of the statements inside the class (so that the header of the main method is indented by one tab stop, for instance) as well as all of the statements inside the main method. Since the main method is already indented one tab-stop, that means that the statements inside the body of the main method will be indented by two tab stops.

For a more exhaustive treatment of programming style, make sure you read the CS 170 Programming Styleguide which will be used to evaluate your homework assignments. You can find the style guide on the class Web page.



Comments

- Comments help human readers understand your code
 - Comments are ignored by the compiler
- Java has three kinds of comments
 - **Single line**: start with `//`, end at the end of a line
 - **Multi-line** comments, delimited by `/*` and `*/`
 - **JavaDoc** comments, start with `/**`, end with `*/`
 - Use special tags like `@author` and `@version`
- **Exercise 16**: Format your code and add a JavaDoc file comment to the top of the file.

Slide 17

Comments

Duration: 00:02:36

Advance mode: Auto

Notes:

Comments are lines added to your code for the benefit of human readers (such as me, of course, but also for yourself when you can't remember what your code was supposed to do when you return to it next week.) Comments are completely ignored by the compiler.

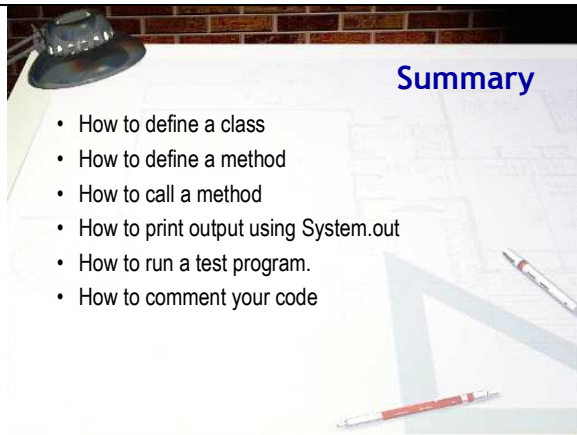
Let's look at each of Java's three types of comments.

Lines that begin with two forward slashes, `//`, are called single-line comments. Both the slashes and anything following them on the same line are ignored by the compiler. (Make sure you don't insert a space between the two slashes, however, because that confuses the compiler). You can have regular program code on the same line as a single-line comment, provided it appear before the comment.

You can also create comments that span multiple lines by using the delimiter `/*` (a forward slash, immediately followed by an asterisk) to start the comment, and the delimiter `*/` (an asterisk immediately followed by a forward slash) to end the comment. You cannot have multi-line comments inside other multi-line comments, but you can put any number of single-line comments inside a multi-line comment.

A third type of comment is called a JavaDoc or Documentation comment. These comments begin with the symbol `/**`, and end with a `*/` just like a regular multi-line comment. These special comments are used to automatically create Java documentation, using the javadoc program. There are several types of JavaDoc comments that you'll learn about throughout the semester. For right now, let's look at the introductory comment that appears at the beginning of every file. Open the CS 170 Style Guide from the class Web page, turn to page 2 and notice the example JavaDoc file comment. JavaDoc comments use the `@` symbol to create specific tags used to generate documentation. Every file you create should start with a JavaDoc comment using the `@author` and the `@version` tags to identify yourself and the date and version of the program you're working on.

For Exercise 16, format `TicTacToeBoardPrinter.java` using the suggestions on the previous page, and then follow the pattern shown in the style guide to add a JavaDoc file comment to the top of your program file. Shoot me a screenshot of your code.



Slide 18

Summary

Duration: 00:00:49

Advance mode: Auto

Notes:

Well, that's it for this lecture. Let's review the most important things we covered.

First, you learned the basic syntax for defining a class, including braces, keywords and identifiers.

Next, you learn the syntax for defining a method, including access specifiers, the return type and parameters. To do this, you added a `main()` method to your program.

You also learned how to call or invoke a method, calling the `print` and `println` methods using the `System.out` object.

You learned how to test your code by running the test program that I supplied.

You learned how to comment and format your code, using the CS 170 style guide to help you.